

Licenciatura em Engenharia Informática

Sistemas Distribuídos

Trabalho Prático 2

Hugo Paredes | Tiago Pinto | Cristiano Pendão

João Nogueira al81605

Luis Carvalho al81967

Vila Real, junho 2026

1. Introdução

O conceito *One Health* estabelece a forte interligação entre a saúde humana, animal e ambiental, sublinhando a importância da monitorização de fatores urbanos como qualidade do ar, temperatura, humidade, ruído e luz UV para a deteção de indicadores relevantes na identificação de riscos para a saúde pública.

Para dar resposta a este desafio, o Trabalho Prático 2 estende a infraestrutura desenvolvida no TP1, elevando-a a um cenário de produção distribuído e heterogéneo. A plataforma *UrbanHealth* assenta agora numa arquitetura *Edge-to-Cloud* composta por quatro camadas funcionais: Sensores IoT, Gateways, Microserviços em Python e um Servidor Central. Este relatório documenta as opções arquitetónicas, o protocolo de comunicação adotado e as tecnologias que garantem a fiabilidade, a escalabilidade e a observabilidade do sistema.

2. Arquitetura do Sistema

O sistema divide-se em quatro camadas funcionais que formam uma pipeline contínua de recolha, pré-processamento, transporte e análise de dados:

Sensores

Cada sensor é um container Docker que simula um dispositivo IoT instalado numa zona urbana. Recolhe leituras de temperatura (TEMP), humidade (HUM), dióxido de carbono (CO2), ruído (NOISE), partículas em suspensão (PM2), ultravioleta (UV) e fluxos de vídeo (VIDEO) à frequência configurada no ficheiro JSON. A comunicação com o gateway é feita de forma assíncrona via *RabbitMQ* para dados ambientais e diretamente por *UDP* para os frames de vídeo, minimizando a latência nos fluxos binários.

Gateways

O gateway opera como nó intermédio. Recebe as leituras dos sensores, invoca o microserviço *python-preprocess* via *gRPC* para normalização e validação dos dados, e reencaminha os resultados ao Servidor via TCP (dados ambientais) e UDP (vídeo). Para garantir a tolerância a falhas, o gateway implementa o padrão *Store-and-Forward*: caso a ligação ao servidor seja perdida, os dados são armazenados localmente numa base *SQLite* e re-enviados automaticamente quando a conectividade é restabelecida.

Microserviços Python (gRPC)

Dois microserviços independentes complementam a lógica de negócio do sistema. O *python-preprocess*, co-localizado com o gateway na rede Docker *urban-edge*, recebe cada leitura bruta e aplica normalização e validação. O *python-analysis*, co-localizado com o servidor na rede *urban-cloud*, realiza análise estatística sobre os dados já persistidos. Ambos os serviços expõem contratos formais definidos em *Protocol Buffers* (ficheiros .proto) e implementam verificações de saúde (*healthcheck*) que bloqueiam o arranque dos seus clientes até estarem prontos.

Servidor Central

O servidor central é o destino final da pipeline. Recebe os dados processados pelos gateways, invoca o *python-analysis* para análise agregada, persiste tudo numa base de dados *SQLite* e serve um *dashboard web* em tempo real acessível na porta 8081 (configurável). A persistência dos dados é garantida por um volume Docker montado em `database_data/___`, assegurando que a informação sobrevive a reinicializações do container.

3. Protocolo de Mensagens

A comunicação entre as entidades do sistema assenta numa estrutura de mensagem universal implementada na biblioteca partilhada *Shared* (C#). Cada mensagem é composta por um cabeçalho descritivo em texto plano e um corpo binário opcional, permitindo transportar tanto métricas escalares como fotogramas de vídeo no mesmo formato.

O cabeçalho contém quatro campos obrigatórios: o comando da operação (CMD), o identificador do sensor (SID), o identificador do gateway (GID) e a *timestamp* ISO 8601. Adicionalmente, suporta um dicionário flexível de pares chave-valor, permitindo a injeção dinâmica de parâmetros como a zona geográfica, o tipo de métrica e o valor lido. O cabeçalho termina com o marcador explícito `<EOF>`, garantindo a delimitação correta dos pacotes e prevenindo a corrupção de leitura nas streams TCP.

Para a transmissão de vídeo, os frames são fragmentados em segmentos de até 1200 bytes (configurável) e enviados por UDP. O *VideoFrameAssembler* no destino reconstrói cada fotograma a partir dos fragmentos recebidos, descartando fotogramas incompletos após um TTL configurável. Este mecanismo garante baixa latência de vídeo sem sobrecarregar as streams TCP com dados binários volumosos.

4. Escolhas de Implementação

O desenvolvimento do TP2 implicou diversas decisões técnicas que evoluíram as soluções do TP1, adaptando-as a um cenário distribuído multi-host e heterogéneo.

Mensageria Assíncrona com RabbitMQ

No TP1, os sensores comunicavam diretamente com o gateway via TCP. No TP2, adotou-se o *RabbitMQ* como broker de mensagens centralizado, desacoplando produtores de consumidores. Os sensores publicam as suas leituras num *exchange* de topo (`urbanhealth_exchange`), e os *gateways* *subscvem* com *routing keys específicas*. Esta abordagem elimina a dependência de ligações TCP diretas, suporta vários sensores a publicar em simultâneo e simplifica a incorporação de novos tipos de sensores sem alterar os gateways.

Microserviços gRPC em Python

Para separar as responsabilidades de processamento do código C# principal, introduziram-se dois microserviços Python que comunicam com o respetivo componente C# por *gRPC*. A escolha do *gRPC* sobre REST justifica-se pela eficiência binária do *Protocol Buffers*, pela tipagem forte dos contratos e pela geração automática de *stubs* cliente e servidor. Os contratos estão formalizados nos ficheiros *.proto* em *Contracts/*, garantindo uma fonte única de verdade para ambas as linguagens.

Orquestração com Docker Compose

O sistema é inteiramente containerizado e orquestrado por quatro ficheiros Docker Compose independentes, *broker*, *server*, *gateway* e *sensor*, que podem ser lançados separadamente em máquinas distintas. Esta separação permite testar cada camada de forma isolada e distribuí-la por vários hosts físicos usando IPs reais nos ficheiros de configuração JSON, sem qualquer alteração ao código. O *start.bat* oferece um painel de controlo interativo que orquestra o arranque sequencial e monitoriza o estado de cada serviço antes de avançar para o seguinte.

Configuração por JSON e Alias local

Cada componente, sensor, gateway e servidor, lê a sua configuração a partir de ficheiros JSON independentes. O *EndpointResolver* partilhado resolve automaticamente o alias *local*: dentro de Docker traduz para *host.docker.internal* e fora traduz para *127.0.0.1*. Este mecanismo permite simular o sistema completo numa única máquina sem alterar configurações, e adaptar facilmente a uma topologia multi-host substituindo *local* pelo IP real da máquina de destino.

Tolerância a Falhas e Store-and-Forward

Mantendo a solução do TP1, cada gateway contém um *LocalCacheManager* que persiste localmente as leituras não enviadas numa base *SQLite*. Quando a ligação ao servidor é restabelecida, o gateway drena automaticamente a cache e retransmite os dados em atraso. Adicionalmente, o gateway mantém um *heartbeat* periódico para cada sensor registado, atualizando o seu estado para indisponível caso deixe de receber sinais dentro do limiar configurado.

5. Tecnologias Utilizadas

A tabela seguinte resume a stack tecnológica adotada em cada camada do sistema:

Camada	Tecnologia
Sensores, Gateway e Servidor	C# / .NET
Microserviços de processamento	Python 3
Comunicação Sensor → Gateway	RabbitMQ + UDP (vídeo)
Comunicação Gateway → Servidor	TCP + UDP (vídeo)
Comunicação com microserviços	gRPC / Protocol Buffers
Base de dados	SQLite
Containerização	Docker + Docker Compose
Dashboard web	HTML/JS (servido pelo servidor)

6. Configuração de Rede e Portas

O sistema adota uma configuração híbrida de comunicação. Os pares Sensor→Gateway, Sensor/Gateway→RabbitMQ e Gateway→Servidor usam IPs configurados nos ficheiros JSON, passando pelas portas expostas do host. Os pares gRPC (Gateway→python-preprocess e Servidor→python-analysis) resolvem por DNS interno Docker, sendo os únicos que dependem da rede de containers. A tabela seguinte lista as portas do sistema, num cenário predefinido (2 sensores + 2 gateways):

Serviço	Porta	Protocolo
Servidor Central (TCP)	5001	TCP
Servidor Central (Vídeo)	5003	UDP
Dashboard Web	8081	HTTP
Gateway G101 (UDP)	5004	UDP
Gateway G101 (Preview)	8080	HTTP
Gateway G102 (UDP)	5005 (host)	UDP
Gateway G102 (Preview)	8082	HTTP
RabbitMQ (AMQP)	5672	TCP
RabbitMQ (Web)	15672	HTTP
python-preprocess (gRPC)	50051	gRPC
python-analysis (gRPC)	50052	gRPC

7. Conclusão e Trabalho Futuro

O projeto *UrbanHealth TP2* materializou com sucesso a evolução da infraestrutura do TP1 para um sistema verdadeiramente distribuído e heterogéneo. A introdução do *RabbitMQ* desacoplou os sensores dos gateways, os microserviços *gRPC* em Python separaram as responsabilidades de processamento, e a orquestração por Docker Compose facilitou o deployment tanto em modo local como multi-host. O padrão *Store-and-Forward* e o *heartbeat* periódico garantem a resiliência do pipeline perante falhas temporárias de conectividade.

Como propostas de desenvolvimento futuro, destaca-se a integração de certificados *TLS* nos sockets TCP e no broker *RabbitMQ*, assegurando a confidencialidade dos dados em trânsito. No plano analítico, pretende-se explorar modelos preditivos de *Machine Learning* no microserviço de análise, permitindo detetar anomalias ambientais e lançar alertas preventivos de forma proativa, para isso deixando de gerar dados aleatórios nos sensores e passando para uma simulação de telemetria real. Por fim, a migração do Docker Compose para um orquestrador como o *Kubernetes* permitiria escalar horizontalmente cada camada de forma elástica, acompanhando o crescimento do tráfego de dados em cidades inteligentes.

8. Anexos

Repositório GitHub: <https://github.com/LuisCarvalhoGit/SD-TP2>

Guia de Deployment detalhado: [UrbanHealth/DEPLOYMENT.md](#)

Guia de configuração de IPs e portas: [UrbanHealth/IP_CONFIGURATION_GUIDE.md](#)